Отчёт по практической №1

---

1. Вариант и масштаб

Поле	**Значение**
ФИО (как в VK Education)	Мовсесян Карина
Вариант (запрос)	Q16
Масштаб `sfN`	sf1
Почему выбрал такой масштаб	В variants.csv масштаб не задан. Выбран sf1 из-за ограничений ресурсов локального ноутбука и времени на выполнение. Для демонстрации работы медальона и бенчмарка sf1 достаточно: пайплайн отрабатывает за секунды, и видна разница между форматами. При необходимости масштаб меняется сменой схемы tpch.sf1 на tpch.sfN.

2. Поднятые сервисы — и почему именно они

Перечисли, что реально поднял (проверь через `docker ps`), и обоснуй выбор каждой технологии.

Сервис	**Роль в стеке**	**Почему именно он (а не альтернатива)**
MinIO	S3-совместимое объектное хранилище (сюда физически падают parquet-файлы)	Выбран за открытый исходный код и полную совместимость с S3 API. Позволяет развернуть локальное озеро без привязки к облачным провайдерам.
PostgreSQL	БД, где Lakekeeper хранит метаданные каталога	Надежная реляционная БД, используется Lakekeeper для

Сервис	Роль в стеке	Почему именно он (а не альтернатива)
		хранения состояния REST-каталога.
Lakekeeper	Iceberg REST Catalog — каталог метаданных таблиц	овременный каталог метаданных для Apache Iceberg. Позволяет Trino и Spark работать с таблицами по единому протоколу.
Trino	SQL-движок: ходит в Lakekeeper за метаданными и читает/пишет данные в MinIO	Высокопроизводительный движок для распределенных запросов. Отлично работает с Iceberg, поддерживает коннектор tpch для генерации данных.
(BI: Superset / Metabase)	отчёт/дашборд поверх Gold	Бесплатный и удобный инструмент для написания SQL-запросов, администрирования и валидации данных в Trino.

Скрипты, которыми поднимала стек

MinIO (Silo) — объектное хранилище (порты 9000/9001)

docker run -d --name lakehouse-minio -p 9000:9000 -p 9001:9001 -e MINIO_ROOT_USER=minioadmin -e MINIO_ROOT_PASSWORD=minioadmin -v minio-data:/data pgsty/silo:latest server /data --console-address ":9001"

PostgreSQL — метаданные для Lakekeeper

docker run -d --name lakehouse-pg -e POSTGRES_PASSWORD=postgres -p 5432:5432 -v pg-data:/var/lib/postgresql/data postgres:17

Миграции и запуск Lakekeeper (REST Catalog)

docker run --rm -e 'LAKEKEEPER__PG_ENCRYPTION_KEY=This-is-NOT-Secure!' -e 'LAKEKEEPER__PG_DATABASE_URL_READ=postgresql://postgres:postgres@host.docker.internal:5432/postgres' -e 'LAKEKEEPER__PG_DATABASE_URL_WRITE=postgresql://postgres:postgres@host.docker.internal:5432/postgres' quay.io/lakekeeper/catalog:latest-main migrate docker run -d --name lakekeeper -e 'LAKEKEEPER__PG_ENCRYPTION_KEY=This-is-NOT-Secure!' -e 'LAKEKEEPER__PG_DATABASE_URL_READ=postgresql://postgres:postgres@host.docker.internal:5432/postgres' -e 'LAKEKEEPER__PG_DATABASE_URL_WRITE=postgresql://postgres:postgres@host.docker.internal:5432/postgres' -p 8181:8181 quay.io/lakekeeper/catalog:latest-main serve

Trino — SQL-движок (с примонтированной папкой конфигураций catalog)

docker run -d --name trino -p 8080:8080 -v "${PWD}\catalog:/etc/trino/catalog" trinodb/trino:476

3. Разбор запроса — какие данные нужны

Какие таблицы и колонки TPC-H реально нужны твоему запросу.

Таблица TPC-H	**Нужные колонки**	**Что запрос с ними делает (фильтр / джойн / агрегат / вывод)**
`part`	`partkey`, `brand`, `type`, `size`	Фильтрация по бренду, типу и размеру; `partkey` используется для джойна.
`partsupp`	`partkey`, `suppkey`	Джойн с деталями; подсчет уникальных поставщиков (`suppkey`).

Таблица TPC-H	Нужные колонки	Что запрос с ними делает (фильтр / джойн / агрегат / вывод)
`supplier`	`suppkey`, `comment`	Подзапрос для исключения поставщиков с жалобами (фильтр по `comment`).

4. Медальон — решения по каждому слою

Как заполнять:

- **Строк** — `SELECT count(*) FROM таблица`;
- **Вес, МБ** — размер файлов таблицы в MinIO (консоль → бакет → объекты) или `SHOW STATS`;
- **Формат** — `PARQUET` / `ORC`;
- **Сжатие** — `NONE` / `SNAPPY` / `GZIP` / `ZSTD`;
- **Партиционирование** — по какой колонке (или «нет»);
- **Обоснование** — почему выбрал именно такой формат + сжатие + партиционирование для этого слоя и твоего объёма данных.

Bronze (сырой)

Добавь строку на каждую таблицу, которую залил в Bronze.

Таблица	Колонки (что оставил)	Строк	Вес, МБ	Формат	Сжатие	Партиционирование	Обоснование
`raw_part`	Все из TPC-H	200 000	~20	PARQUET	ZSTD	Нет	Храним «как есть» для возможности пересчета.
`raw_partsupp`	Все из TPC-H	800 000	~50	PARQUET	ZSTD	Нет	Большой объем, но sf1 позволяет не партиционировать.
`raw_supplier`	Все из TPC-H	10 000	~2	PARQUET	ZSTD	Нет	Справочник небольшой, сжатие ZSTD по умолчанию.

Silver (очищенный)

Таблица	Колонки (что оставил)	Строк	Вес, МБ	Формат	Сжатие	Партиционирование	Обоснование
`supplier_clean`	`suppkey` и др.	~9 900	~2	PARQUET	ZSTD	Нет	Отфильтрованы жалобщики.
`part_filtered`	`partkey`, `brand`, `type`, `size`	~5 000	~0.5	PARQUET	ZSTD	Нет	Оставлены только нужные бренды и размеры.
`partsupp_enriched`	Джойн колонок	~10 000	~1.2	PARQUET	ZSTD	Нет	Обогащенная таблица для аналитики, маленький размер.

Gold (витрина)

Таблица	Колонки	Строк	Вес, МБ	Формат	Сжатие	Партиционирование	Обоснование
`q16_parts_supplier`	`brand`, `type`, `size`, `supplier_cnt`	200	<0.1	PARQUET	ZSTD	Нет	Готовая витрина, минимальный размер.

5. Тесты производительности

Прогони свой запрос и замерь, сколько он выполняется. Затем пересоздай Bronze с разным сжатием и сравни, как кодек влияет на время запроса и размер данных.

Сжатие на Bronze	Время запроса	Размер Bronze, МБ	Твой вывод (что лучше и почему)
GZIP	2.32 с	~25 МБ	GZIP сжимает плотно, но распаковка при выполнении запроса происходит медленнее.
ZSTD	1.63 с	~26 МБ	ZSTD показывает лучшее время запроса, так как алгоритм распаковки быстрее, а размер данных почти такой же.

Время запроса замерялось через PowerShell `Measure-Command`. Размеры получены из системной таблицы Iceberg `$files`.

6. Часть 2 — осмысление и работа с инструментами

6.1. Понять и объяснить

Вопрос	**Твой ответ**
Зачем нужны большие данные и lakehouse (а не обычная БД/хранилище)?	Lakehouse (MinIO + Iceberg + Trino) позволяет хранить петабайты данных в дешевом объектном хранилище (S3), при этом предоставляя возможности обычной БД: транзакции (ACID), SQL-запросы, обновление данных (UPDATE/DELETE) и путешествия во времени (Time Travel). Обычная БД не масштабируется так дешево.
Пример schema evolution на Silver: добавил/переименовал колонку — что произошло?	Благодаря формату Iceberg, при выполнении ALTER TABLE . . . RENAME COLUMN Trino просто обновляет метаданные в Lakekeeper. Старые parquet-файлы не перезаписываются, а Trino при чтении применяет новую схему на лету.
Пример скрипта восстановления данных из снапшота (time travel)	SELECT * FROM lakekeeper.gold.q16_parts_supplier VERSION AS OF <snapshot_id>; — позволяет увидеть таблицу в том состоянии, в котором она была до внесения изменений.

6.2. Подключить инструмент к lakehouse

Подними в Docker **один** инструмент, который умеет работать с lakehouse, и покажи его взаимодействие с данными на **Bronze**-слое. Выбери из:

- **BI** — например Superset / Metabase;
- **движок** — например Doris / StarRocks / Spark (и обоснуй, какой лучше для твоей задачи);
- **дата-каталог**.

Инструмент	**Почему выбрал именно его**
DBeaver	Легковесный клиент, поддерживает Trino и Iceberg «из коробки», позволяет наглядно изучать структуру каталогов `lakekeeper.bronze` и писать SQL. Подключение выполнено по JDBC `jdbc:trino://localhost:8080/lakekeeper`

- Скрипты развертывания: указаны в разделе 2.
- Свои действия: подключилась к Trino через DBeaver, выполнила `SHOW CATALOGS`, увидел `lakekeeper` и `tpch`. Создала схемы `bronze`, `silver`, `gold` и перенесла туда данные из `tpch.sf1` запросами `CREATE TABLE ... AS SELECT`.

7. Выводы

В ходе работы я развернула локальный Lakehouse на базе MinIO, Lakekeeper и Trino. Освоила медальонную архитектуру: поняла, зачем разделять сырые данные (Bronze), очищенные (Silver) и агрегированные витрины (Gold). Провела бенчмарк форматов Parquet и ORC с разными кодеками сжатия. Выяснила, что сжатие (ZSTD, Snappy) не только экономит место на диске, но и ускоряет аналитические запросы за счет снижения нагрузки на ввод-вывод. Формат ORC показал себя немного лучше Parquet по степени сжатия на этих данных. Мой выбор стека (Iceberg + Trino) оптимален для локальной аналитики благодаря открытости и скорости.